level method and using it in an instance method we're defining. This is a common pattern when creating these kinds of class-level macros.

In addition to passing parameters from the class method into the body of the method being defined, we can also use the parameter to dynamically determine the name of the method or methods to create. Here's an example that creates a new kind of attr_accessor that logs all assignments to a given instance variable:

metaprogramming/accessor_1.rb

```
class AttrLogger
  def self.attr_logger(name)
    attr_reader name

    define_method(:"#{name}=") do |val|
      puts "Assigning #{val.inspect} to #{name}"
      instance_variable_set("@#{name}", val)
    end
  end
end

class Example < AttrLogger
  attr_logger :value
end

ex = Example.new
ex.value = 123
puts "Value is #{ex.value}"
ex.value = "cat"
puts "Value is now #{ex.value}"
```

produces:

```
Assigning 123 to value
Value is 123
Assigning "cat" to value
Value is now cat
```

Again, we use the fact that the block defining the method body is a closure, accessing the name of the attribute in the log message string. Notice we also make use of the fact that attr_reader is simply a class method—we can call it inside our class method to define the reader method for our attribute. Note another common bit of metaprogramming—we use instance_variable_set to set the value of an instance variable. The argument to instance_variable_set is possibly unexpected—it's a string starting with @ containing the name of the variable you want to set. There's a corresponding instance_variable_get method that fetches the value of a named instance variable.

Class Methods and Modules

You can define class methods in one class and then use them in subclasses of that class. But it is often inappropriate to use subclassing for the kinds of metaprogramming we have been showing, either because we already have to subclass some other class or because our design aesthetic rebels against making Song a subclass of Logger.

In these cases, you can use a module to hold your metaprogramming implementation. As we've seen, using extend inside a class definition will add the methods in a module as class methods to the class being defined: